ПРАКТИКА ПОСЛЕ REACT-API

Лабораторная работа 6

Роутинг, общее состояние и кеш запросов в React

Продолжите **свой проект из лабораторной №5**. Добавьте страницы, общий клиентский выбор и кеш серверных данных. Используйте сущности своей темы: товары, фильмы, задачи и т. п.

Стек: React Router, Redux Toolkit, React Redux и RTK Query. Вариант на Zustand + TanStack Query реализовывать не требуется.

```
npm install react-router @reduxjs/toolkit react-redux
```

КРИТЕРИИ Накопительные: оценка 4 включает все условия на 3, оценка 5 включает все условия на 4. Срок сдачи назначает преподаватель.

3 / Удовлетворительно

Маршруты. Работают три страницы: список объектов, детали по динамическому **id** и отдельная страница выбранных объектов. Переходы через **Link** или **NavLink** не перезагружают документ.

Общее состояние. Redux store создан один раз вне переключаемых страниц и подключён через **Provider**. **createSlice** хранит корзину, избранное или другой клиентский выбор. Есть добавление через **useDispatch**; минимум два компонента читают общее состояние через **useSelector**.

Переходы. Выбор сохраняется при смене страницы. Сохранение после полной перезагрузки браузера не требуется.

API и кеш. Минимум один список загружается **реальным HTTP-запросом** через RTK Query. Подключены **createApi**, его reducer и middleware. Компонент использует сгенерированный query-хук.

Отображение. Показаны первая загрузка, сообщение об ошибке и успешный список со стабильным **key**.

НА ЗАЩИТЕ Добавьте объект, покажите общий счётчик, смените страницу и загрузите данные API. Объясните, где хранится клиентский выбор и где хранится ответ сервера.

4 / Хорошо

Выполнены все условия на 3, а также:

Навигация. Общая шапка и страницы организованы через layout и **Outlet**. Неизвестный путь и неизвестный **id** показывают понятное сообщение. Работают «Назад» и «Вперёд» браузера.

Прямая ссылка. Детали открываются по скопированному адресу и после обновления страницы. Для GitHub Pages подходит **HashRouter**; для **BrowserRouter** хостингу нужен fallback на **index.html**. Проверьте базовый путь ресурсов в подпапке.

Управление выбором. Можно убрать объект или изменить количество. Все потребители сразу видят результат. Счётчики и суммы вычисляются из исходных данных без независимых копий одного значения в нескольких **useState**.

Состояния запроса. После ошибки доступен повтор без перезагрузки. Для успешного пустого ответа есть отдельное сообщение. Первая загрузка и фоновое обновление различаются через **isLoading** / **isFetching**; при обновлении прежние данные остаются видны.

Проверка кеша. Покажите два компонента с одинаковым endpoint и аргументом либо возврат к использованному списку. Во вкладке **Network** объясните, когда используется кеш и когда отправляется запрос. Ответ API не копируется вручную в отдельный slice.

НА ЗАЩИТЕ	Покажите ошибку, повтор и результат; пустой список и повторное использование кеша. Объясните, почему другой id может означать другую запись кеша.

Готовый HTTP API

Если ваше API не сохраняет изменения, используйте **выдаваемую отдельно папку lab-api**: **server.mjs**, **books.json** и **README.md**. Сервер уже написан; разрабатывать backend не нужно. Нужен Node.js 22 или новее.

Скопируйте папку целиком в корень своего проекта. В первом терминале запустите сервер, во втором запустите React-приложение:

```
node lab-api/server.mjs
```

В **fetchBaseQuery** задайте адрес:

```
baseUrl: 'http://127.0.0.1:4191/api/'
```

Сервер хранит изменения до перезапуска. В **lab-api/README.md** описаны запросы, ошибка, пустой ответ и сброс. Для своей темы можно изменить **books.json**, сохранив поля **id** и **price**. Сервер запускается локально; GitHub Pages его не выполняет.

5 / Отлично

Выполнены все условия на 4, а также:

Изменение данных. Форма или действие отправляет RTK Query mutation: например, меняет цену, название или статус. HTTP-метод соответствует API: **POST**, **PUT** или **PATCH**.

Валидация. Некорректное значение не отправляется. Например, цена должна быть конечным числом не меньше нуля; пустое поле не превращается незаметно в ноль.

Результат сохранения. Виден статус ожидания, повторная отправка временно недоступна. Ошибка показана пользователю; после неё можно повторить попытку. Успех проверяется по ответу запроса. Для **try/catch** с mutation trigger используйте **.unwrap()**.

Обновление кеша. Query endpoints используют **providesTags**, успешная mutation использует **invalidatesTags**. Новые данные появляются без перезагрузки. Проверьте список и детали, если у них отдельные endpoints.

Разделение данных. Черновик формы остаётся локальным, клиентский выбор хранится в своём slice, ответы сервера хранятся в RTK Query. Другие экраны не показывают вручную сохранённую старую копию.

НА ЗАЩИТЕ

Покажите изменение → mutation → успешный ответ → обновление затронутых query → новый результат. Воспроизведите ошибку сохранения и повтор. Объясните invalidation для активной и неиспользуемой записи кеша. Общего тега достаточно; точечные теги и оптимистические обновления не обязательны.

API должно сохранять запись: последующий GET возвращает изменённое значение. Подходит готовый lab-api со стр. 2. Локальный массив с Promise вместо HTTP не заменяет API. База данных, авторизация и отдельная серверная платформа не требуются.

Что сдать

1. Репозиторий. Ссылка на исходники с инструкцией запуска в **README**.

2. Приложение. Работающий локальный вариант или хостинг, если публикация предусмотрена преподавателем.

3. Краткое описание. Новые страницы, общее состояние, endpoints и выполненный уровень 3 / 4 / 5.

4. Сценарий проверки. Как воспроизвести загрузку, ошибку и пустой ответ; для оценки 5 также изменение данных. Подойдут настройки учебного сервера, DevTools или подготовленные данные.

На защите объясните путь данных своими словами. Не размещайте секретные ключи в клиентском коде.